MMC204– SOFTWARE ENGINEERING

MODULE- I

Syllabus: Introduction to Software Engineering, Software Process, Perspective and Specialized, Process Models –Introduction to Agility-Agile process-Extreme programming-XP Process- Case Study.

1. Define a system and computer-based system.

Ans:

System: In its most basic form, a system can be understood as consisting of elements, interconnections, and a function or purpose. These elements are interconnected and work together to achieve a common goal.

Computer-based System: These systems leverage computers to process data, store information, and perform computations to achieve specific objectives. They typically involve hardware (physical components) and software (programs) working in tandem.

Examples:

- **Computer system:** A personal computer (PC) with its hardware (keyboard, monitor, CPU) and software (operating system, applications).
- **Computer-based information system:** An electronic health record system that utilizes computers to store and manage patient data.

2. What is Software Engineering? /Define software engineering.

Ans: Software engineering is a field that applies engineering principles and best practices to design, develop, test, and maintain software applications. It involves a systematic and disciplined approach to building high-quality, reliable, and maintainable software solutions that meet user needs.

3. What is Software? List the characteristics.

Ans: Software is a collection of instructions, data, or programs that tell a computer what to do. It's an intangible, non-physical component that enables computers to perform various tasks. Key characteristics of software include functionality, dependability, efficiency, usability, maintainability, and scalability.

Characteristics:

1.Intangible: Software cannot be physically touched or seen, unlike hardware components like a mouse or keyboard.
2. Non-perishable: Unlike physical goods, software does not degrade or wear out over time.
3. Easy to replicate: Software can be easily copied and distributed, unlike physical goods that require manufacturing and shipping.
4. Complex: Software can be intricate, with many interconnected parts and features, making it challenging to understand and modify, especially in large and complex systems.
5. Changing requirements: Software may need to be updated or modified as user needs and technology evolve.
6. Bugs and issues: Software can have errors or bugs that need to be identified, tested, and corrected.
7. Functionality: This refers to the degree to which the software performs its intended tasks.
8. Usability: A good software system should be easy to learn and use, with a user-friendly interface.
9. Efficiency: Software should utilize system resources efficiently, such as processor time and memory, to minimize processing time.
10. Reliability: Reliable software should operate consistently and accurately, minimizing errors and failures.
11. Maintainability: This refers to the ease with which software can be modified, corrected, or updated.
12. Portability: The ability of software to be transferred and run on different hardware and operating systems with minimal changes.

13. Flexibility: The capacity of software to adapt to changing requirements and new environments.
14. Scalability: The ability of software to handle increased workloads or user demands without significant performance degradation

4. What are two types of software products?

Ans: The two main types of software products are generic software and customized software. Generic software is designed for a broad range of users and performs general-purpose tasks, while customized software is built to meet the specific needs of a particular client or organization.

- **Generic Software:** This type of software is designed to be used by a large number of users with diverse needs. It offers a standard set of features and functionality that are suitable for a wide range of tasks. Examples include word processors, spreadsheets, and web browsers.
- **Customized Software:** This type of software is developed to meet the unique requirements of a specific client or organization. It is tailored to their specific business processes, workflows, and needs. Examples include enterprise resource planning (ERP) systems, customer relationship management (CRM) systems, and specialized industry applications

5. Write any two software engineering challenges

Ans:Two major challenges in software engineering are keeping up with rapidly evolving technologies and managing changing requirements and project scope. Another significant challenge is ensuring software quality and reliability throughout the development lifecycle, including thorough testing, quality control, and addressing potential vulnerabilities.

1. Keeping Up with Rapid Technological Advancements:

The software development field is constantly evolving, with new programming languages, frameworks, tools, and methodologies emerging frequently. This rapid pace of change necessitates continuous learning and adaptation for software engineers to remain relevant and maintain their skills. Failure to keep up with these advancements can lead to outdated practices, inefficient development processes, and a lack of competitiveness in the industry.

2. Managing Changing Requirements and Project Scope:

Software projects often involve evolving requirements and project scope due to changing user needs, market demands, or unexpected challenges. Effectively managing these changes requires clear communication, adaptable project planning, and the ability to quickly reassess and adjust development efforts. Failure to do so can lead to delays, cost overruns, and potentially compromised software quality.

6. Software doesn't wear out. Justify.

Ans: Software does not wear out in the traditional sense of the term because it is not a physical entity subject to physical wear and tear. It is a collection of code, and that code will remain intact unless actively modified by a human or another program. Unlike hardware, which can deteriorate due to environmental factors, software is not susceptible to these maladies.

- **Software is not a physical entity:**Hardware components are physical and can degrade over time due to factors like dust, vibration, temperature changes, and abuse. Software, on the other hand, exists as digital instructions, stored on a computer's memory or storage devices. These instructions themselves do not physically degrade.
- **Software can be updated and modified:**While software itself doesn't wear out, it can become outdated and require updates to maintain compatibility with newer hardware or software environments. These updates are not a form of wear but rather a process of adaptation and improvement.
- **Software defects are due to errors in the code:**When software fails, it is because of errors in the original code or in the process of converting the design into executable code. These errors can be

fixed through debugging and patching, which are not a form of wear but rather corrections to the software's functionality.

- **No software spare parts:**When a hardware component wears out, it is typically replaced with a spare part. There is no equivalent concept for software; if software fails, it needs to be corrected or replaced with a new version.
- **Software can deteriorate due to technical debt:**While software doesn't wear out physically, it can suffer from a decline in quality over time due to factors like poorly designed code, unresolved bugs, and accumulated technical debt. This is not physical wear but rather a degradation of the software's overall health

7. What is software processes. / Distinguish between process and methods.

Ans: A software process, also known as the Software Development Life Cycle (SDLC), is the structured approach used to develop and maintain software products. It encompasses all activities, from initial planning and design to coding, testing, deployment, and ongoing maintenance. Essentially, it's a roadmap that helps teams manage the complexities of software development and ensure a high-quality result.

In software engineering, a process is a step-by-step structure for managing the creation of software, including requirements, analysis, design, coding, testing, and maintenance. It provides a roadmap for guiding the development stages and helps manage resources and timelines effectively. In contrast, methods are specific approaches or techniques used within the software development process to achieve a particular outcome, such as "Deliver quality software faster".

Feature	Process	Methods
Focus	Overall structure and management of software creation	Specific approaches and techniques within the process
Scope	Encompasses all aspects of software development	Addresses specific tasks or goals within the process
Examples	Software Development Lifecycle (SDLC), Waterfall, Agile	Object-oriented programming, testing frameworks, design patterns

8. Define the terms product and process in software engineering.

Ans: A software product is the final, tangible result of the software development process, while a software process is the set of activities and methods used to create and maintain that product.

Software Product:

1. A software product is the end result of the software development process, and it is the software system that is delivered to the end-user or deployed in an organization.
2. It's the actual software application, system, or tool that is built.
3. Examples include a mobile app, a web application, an operating system, or a software tool.
4. The product's quality is evaluated based on its functionality, reliability, usability, and other attributes.

Software Process:

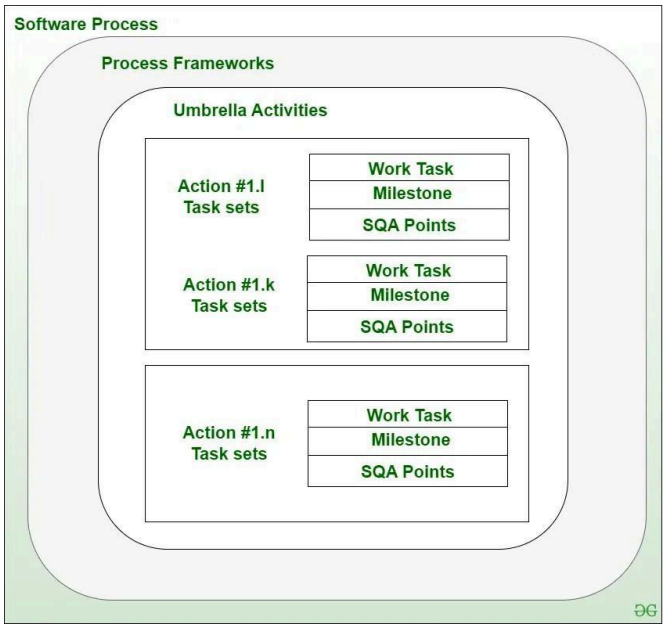
1. A software process refers to the methods, activities, and practices used to develop, maintain, and evolve software.
2. It encompasses the entire lifecycle of a software system, from initial requirements gathering to maintenance and retirement.
3. Examples include various development methodologies like Waterfall, Agile, or DevOps.
4. The process is evaluated based on its efficiency, visibility, flexibility, and ability to deliver high-quality software.
5. It's the "how" of building software, while the product is the "what".

9. Draw Common Process Framework.

Ans: Software Process Framework details the steps and chronological order of a process. Since it serves as a foundation for them, it is utilized in most applications. Task sets, umbrella activities, and process framework activities all define the characteristics of the software development process includes:

1. **Tasks:** They focus on a small, specific objective.
2. **Action:** It is a set of tasks that produce a major work product.
3. **Activities:** Activities are groups of related tasks and actions for a major objective.

Software Process Framework



A software development framework is a structured set of tools, libraries, best practices, and guidelines that help developers build software applications. Think of it as a template or foundation that provides the basic structure and components needed for a software project.

Key Points

1. **Foundation**: It gives a basic structure or template for developing software, so developers don't have to start from scratch.
2. **Components and Tools**: It includes pre-built components and tools that make development faster and easier.
3. **Best Practices and Guidelines**: It offers best practices and guidelines to ensure the software is built in an organized and efficient way.
4. **Customization**: Developers can modify and add new functions to customize the framework to their specific needs.

10. If you have to develop a word processing software product, what process model will you choose? Justify your answer.

Ans:To develop a word-processing software incremental model can be used. Explanation: For software development life cycle, various Software Development Process Models are used . Based on the project motive, the software development process model is selected for development.

The Incremental model is a software Development approach which is used to breakdown the project into smaller and easily manageable parts. In these, each part passes through Requirement, Design, Testing phases and Implementation phase. The overall process continue until we got the complete System.

Incremental Process Model

The Incremental Process Model is a method of software development where the system is built step by step. Instead of delivering the whole system at once, it is developed and delivered in small parts called increments. Each increment builds upon the previous one by adding new functionality, until the complete system is finished.

Key Characteristics of Incremental Process Model

- **Partial System Delivery:** The system is developed and delivered in small, manageable pieces. Each part adds new features to the previous version.
- **Early Functionality:** Basic functionality is available early in the project. This allows users to start using and testing the system quickly.
- **Customer Feedback Loop:** Feedback is collected after each part is delivered. This helps improve the next version of the system.
- **Flexible to Changes:** Changes or new features can be added between increments. This makes the model flexible to evolving needs.
- **Combination of Linear and Iterative Approaches:** Combines the structured approach of Waterfall with flexibility. Supports both planning and ongoing improvements

11. Depict the relationship between work product, task, activity and system.

Ans: The relationship can be visualized as:

- **System** = Activity + Activity + Activity... (e.g., Construction + Modeling + Validation +...)
- **Activity** = Task + Task + Task... (e.g., Code Writing + Database Design + Testing...)
- **Task**: → Work Product (e.g., Code Writing → Code Module)

To build a system, you need to perform a series of activities. Each activity is made up of multiple tasks, and each task produces a work product. The cumulative effect of these work products from the various tasks and activities results in the final system

12. State the benefits and drawbacks of waterfall life cycle model for software development.

Ans: The waterfall model in software development offers structured planning and project management, but lacks flexibility and struggles with complex or evolving requirements.

Benefits:

- **Clear Structure:**

The waterfall model's sequential phases provide a structured and organized framework for software development, making it easier to understand and manage.

- **Early Goal Determination:**

The model helps define the end goal early in the project, enabling better planning and execution.

- **Efficient Information Transfer:**

The sequential nature of the model ensures clear and efficient information transfer between phases.

- **Predictable Results:**

With a well-defined process and clear requirements, the waterfall model can offer more predictable project outcomes.

- **Progress Management:**

The defined phases and tasks make it easy to track progress and manage project timelines.

- **Comprehensive Documentation:**

The model encourages detailed documentation at each stage, providing a historical record and blueprint for the project.

- **Improved Risk Management:**

Rigorous planning reduces risks and allows for the identification and resolution of design issues early in the process.

Drawbacks:

- **Limited Flexibility:**

The waterfall model's rigid structure leaves little room for changes or adjustments during the development process.

- **Difficulty in Addressing Errors:**

If errors or issues arise in a later phase, it can be time-consuming and costly to go back and fix them.

- **Delayed Testing:**

Testing is often delayed until the end of the project, which can be risky and lead to significant delays if major issues are found.

- **Inadequate for Complex Projects:**

The model's simplicity can make it unsuitable for large or complex projects with evolving requirements.

- **Reduced Client Involvement:**

The traditional waterfall model may not adequately involve clients or end-users in the development process

13. What are the pros and cons of Iterative software development models?

Ans: The main reason behind using iterative waterfall model is feedback path. While the feedback loop in the iterative waterfall model primarily focuses on later phases, it's still possible to revisit the feasibility study if changes in requirements or technology emerge, ensuring the project stays aligned with its goals.

Advantages of Iterative Waterfall Model

Following are the advantage of Iterative Waterfall Model:

1. **Phase Containment of Errors**: Errors are detected and fixed as close to their source as possible, reducing costly rework and delays.
2. **Collaboration**: Continuous collaboration between business owners and developers ensures the product meets business needs and improves with feedback at each iteration.
3. **Flexibility**: The model allows for easy incorporation of new requirements or features in subsequent iterations, ensuring the product evolves with the business.
4. **Testing and Feedback**: Regular testing and feedback cycles help identify and fix issues early, improving the product's quality and relevance.
5. **Faster Time to Market**: Incremental development allows parts of the product to be delivered sooner, enabling user feedback while further improvements are made.
6. **Risk Reduction**: Continuous feedback and testing help identify risks early, reducing the likelihood of costly errors and delays.

Drawbacks of Iterative Waterfall Model

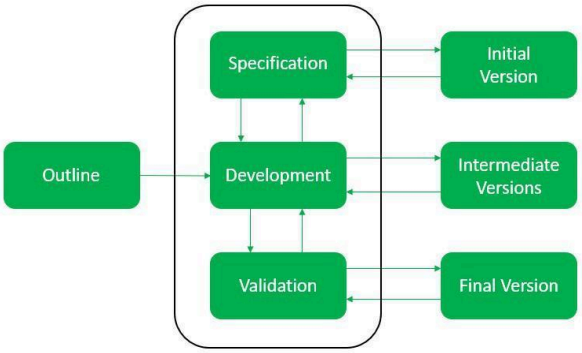
1. **Difficult to incorporate change requests:** The major drawback of the iterative waterfall model is that all the requirements must be clearly stated before starting the development phase. Customers may change requirements after some time but the iterative waterfall model does not leave any scope to incorporate change requests that are made after the development phase starts.
2. **Incremental delivery not supported:** In the iterative waterfall model, the full software is completely developed and tested before delivery to the customer. There is no scope for any intermediate delivery. So, customers have to wait a long for getting the software.
3. **Overlapping of phases not supported:** Iterative waterfall model assumes that one phase can start after completion of the previous phase, But in real projects, phases may overlap to reduce the effort and time needed to complete the project.
4. **Risk handling not supported:** Projects may suffer from various types of risks. But, the Iterative waterfall model has no mechanism for risk handling.
5. **Limited customer interactions:** Customer interaction occurs at the start of the project at the time of requirement gathering and at project completion at the time of software delivery. These fewer interactions with the customers may lead to many problems as the finally developed software may differ from the customers' actual requirements.

14. Define Evolutionary Process Model.

Ans: **Evolutionary Process Model**

The evolutionary model is based on the concept of making an initial product and then evolving the

software product over time with iterative and incremental approaches with proper feedback. In this type of model, the product will go through several iterations and come up when the final product is built through multiple iterations. The development is carried out simultaneously with the feedback during the development. This model has a number of advantages such as customer involvement, taking feedback from the customer during development, and building the exact product that the user wants. Because of the multiple iterations, the chances of errors get reduced and the reliability and efficiency will increase.



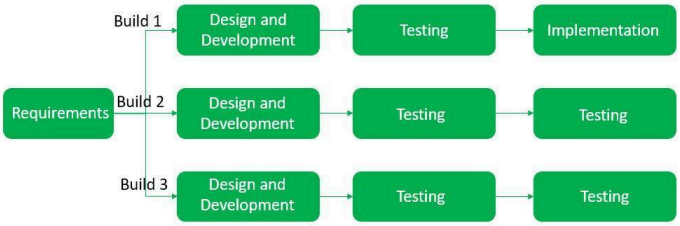
Evolutionary Model

Types of Evolutionary Process Models

1. Iterative Model
2. Incremental Model
3. Spiral Model

Iterative Model

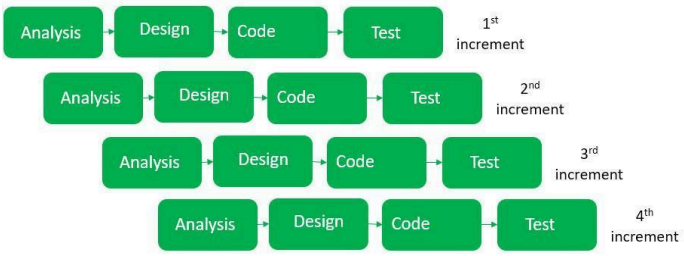
In the iterative model first, we take the initial requirements then we enhance the product over multiple iterations until the final product gets ready. In every iteration, some design modifications were made and some changes in functional requirements is added. The main idea behind this approach is to build the final product through multiple iterations that result in the final product being almost the same as the user wants with fewer errors and the performance, and quality would be high.



Iterative model

Incremental Model

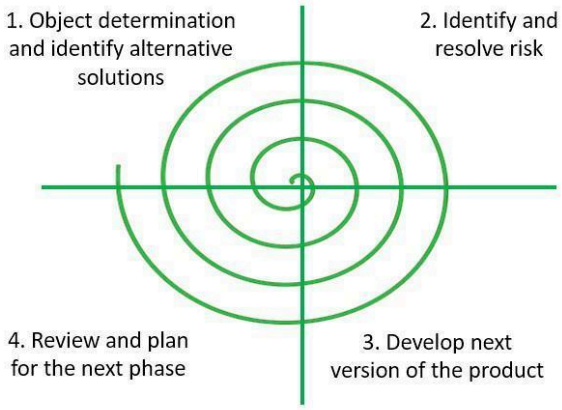
In the incremental model, we first build the project with basic features and then evolve the project in every iteration, it is mainly used for large projects. The first step is to gather the requirements and then perform analysis, design, code, and test and this process goes the same over and over again until our final project is ready.



Incremental Model

Spiral Model

The spiral model is a combination of waterfall and iterative models and in this, we focused on risk handling along with developing the project with the incremental and iterative approach, producing the output quickly as well as it is good for big projects. The software is created through multiple iterations using a spiral approach. Later on, after successive development the final product will develop, and the customer interaction is there so the chances of error get reduced.



15.Compare prototyping approaches in a software process.

Ans: The prototyping approaches in software process are as follows-

[1]. Evolutionary Prototyping: This prototype approach is based on the idea of developing an initial implementation, exposing user commentary, and going through several stages until a sufficient system has been developed as shown in figure.

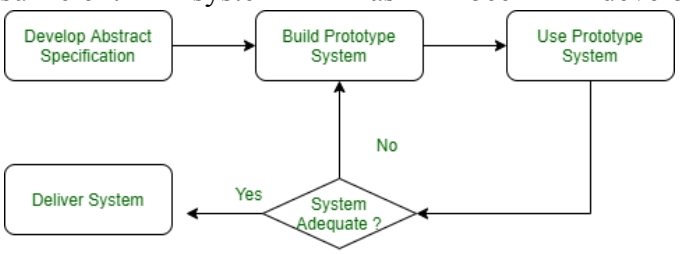


Figure - Evolotionary Prototyping

The advantages to adopting this approach to software development are-

- **[a]. Accelerated Delivery of the System -** Software change is required in the change of business speed. Fast delivery and usability is more important than description of long-term software maintainability functionality.
- **[b]. User Engagement with the System -** The users involvement with the development process is not only to meet their needs, but also means that the system has made a commitment to it and probably wants it to work. There are some problems with evolutionary

prototyping which are particularly important when large, long -lifetime systems are to be developed.

16. What are prototyping model. Mention the advantages & disadvantages?

Ans: is used when the customers do not know the exact project requirements beforehand. In this model, a prototype of the end product is first developed, tested and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.

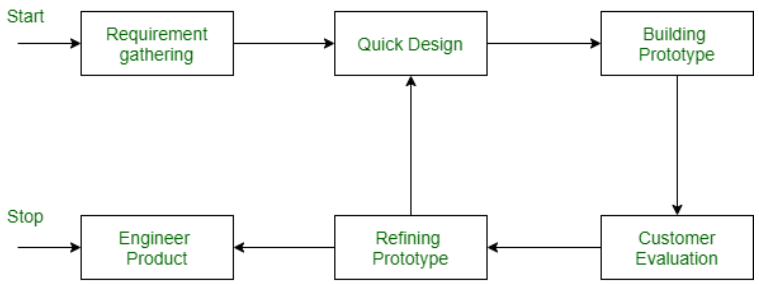


Figure - Prototype Model

System is partially implemented before or during the analysis phase thereby giving the customers an opportunity to see the product early in the life cycle.

Advantages of using Prototype Model :

1. This model is flexible in design.
2. It is easy to detect errors.
3. We can find missing functionality easily.
4. There is scope of refinement, it means new requirements can be easily accommodated.
5. It can be reused by the developer for more complicated projects in the future.
6. It ensures a greater level of customer satisfaction and comfort.
7. It is ideal for online system.
8. It helps developers and users both understand the system better.
9. Integration requirements are very well understood and deployment channels are decided at a very early stage.
10. It can actively involve users in the development phase.

Disadvantages of using Prototype Model :

1. This model is costly.
2. It has poor documentation because of continuously changing customer requirements.
3. There may be too much variation in requirements.
4. Customers sometimes demand the actual product to be delivered soon after seeing an early prototype.

17. What is meant by blocking states in linear sequential model?

Ans: In a linear sequential model, "blocking states" refer to instances where the model or code execution stalls or pauses until a specific event or condition is met before proceeding to the next step in the sequence. This is a common characteristic of sequential models, particularly when implemented with blocking calls, which can lead to various problems, including inflexibility and timing issues.

18. How does "Project Risk" factor affect the spiral model of software

development?

Ans: The Spiral Model of software development is heavily influenced by project risk factors, as it's fundamentally designed to manage and mitigate risks throughout the development lifecycle. The model's iterative nature and focus on risk analysis at each phase allow for early identification and mitigation of potential issues.

Risk factors affect the Spiral Model:

- **Risk Analysis at Every Phase:**

Each loop in the spiral represents a phase where risk analysis is a core activity. Potential risks, such as technical challenges, cost overruns, or schedule delays, are identified and evaluated.

- **Prototyping and Risk Mitigation:**

The spiral model utilizes prototyping at each phase to explore potential solutions and mitigate risks. If a prototype reveals significant risks, the development process can be adjusted or even aborted if the risks are too high.

- **Iterative Nature and Flexibility:**

The spiral model's iterative nature allows for revisiting previous phases and adjusting the development plan in response to identified risks. This flexibility ensures that the project can adapt to changing requirements and evolving risks.

- **Expert Dependency:**

The spiral model relies heavily on experts to identify and analyze risks effectively, and the success of the project depends on the accuracy and completeness of risk analysis.

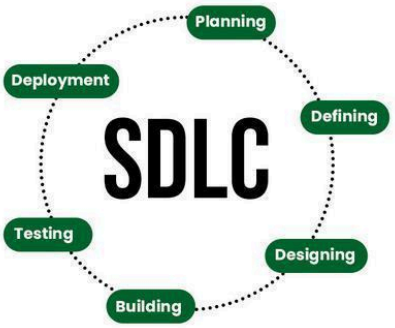
- **Cost Implications:**

Risk analysis and mitigation activities add to the project's overall cost, especially in the early phases. However, the long-term benefits of reduced risks often outweigh the initial investment.

19.Define software life cycle. List all life cycle models and explain all the models in detail with neat diagram. (13 marks)

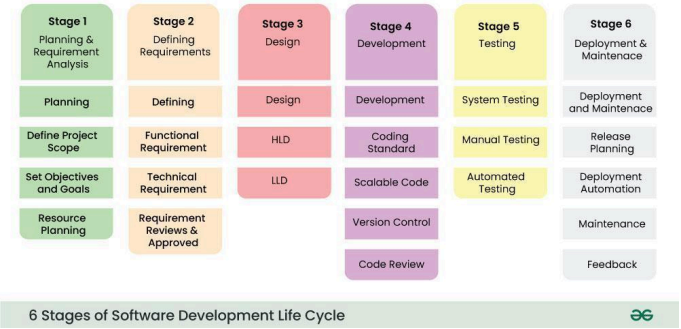
Ans: Software development life cycle (SDLC) is a structured process that is used to design, develop, and test good-quality software. SDLC, or software development life cycle, is a methodology that defines the entire procedure of software development step-by-step. The goal of the SDLC life cycle model is to deliver high-quality, maintainable software that meets the user's requirements. SDLC in software engineering models outlines the plan for each stage so that each stage of the software development model can perform its task efficiently to deliver the software at a low cost within a given time frame that meets users requirements. In this article we will see Software Development Life Cycle (SDLC) in detail.

SDLC is a process followed for software building within a software organization. SDLC consists of a precise plan that describes how to develop, maintain, replace, and enhance specific software. The life cycle defines a method for improving the quality of software and the all-around development process.



Stages of the Software Development Life Cycle

SDLC specifies the task(s) to be performed at various stages by a software engineer or developer. It ensures that the end product is able to meet the customer's expectations and fits within the overall budget. Hence, it's vital for a software developer to have prior knowledge of this software development process. SDLC is a collection of these six stages, and the stages of SDLC are as follows:



Software Development Life Cycle Model SDLC Stages

The **SDLC Model involves six phases or stages** while developing any software.

Stage-1: Planning and Requirement Analysis

Planning is a crucial step in everything, just as in software development. In this same stage, requirement analysis is also performed by the developers of the organization. This is attained from customer inputs, and sales department/market surveys.

The information from this analysis forms the building blocks of a basic project. The quality of the project is a result of planning. Thus, in this stage, the basic project is designed with all the available information.

Stage-2: Defining Requirements

In this stage, all the requirements for the target software are specified. These requirements get approval from customers, market analysts, and stakeholders.

This is fulfilled by utilizing SRS (Software Requirement Specification). This is a sort of document that specifies all those things that need to be defined and created during the entire project cycle.

Stage-3: Designing Architecture

SRS is a reference for software designers to come up with the best architecture for the software. Hence, with the requirements defined in SRS, multiple designs for the product architecture are present in the Design Document Specification (DDS).

This DDS is assessed by market analysts and stakeholders. After evaluating all the possible factors, the most practical and logical design is chosen for development.

Stage-4: Developing Product

At this stage, the fundamental development of the product starts. For this, developers use a specific programming code as per the design in the DDS. Hence, it is important for the coders to follow the protocols set by the association. Conventional programming tools like compilers, interpreters, debuggers, etc. are also put into use at this stage. Some popular languages like C/C++, Python, Java, etc. are put into use as per the software regulations.

Stage-5: Product Testing and Integration

After the development of the product, testing of the software is necessary to ensure its smooth execution. Although, minimal testing is conducted at every stage of SDLC. Therefore, at this stage, all the probable flaws are tracked, fixed, and retested. This ensures that the product confronts the quality requirements of SRS.

Documentation, Training, and Support: Software documentation is an essential part of the software development life cycle. A well-written document acts as a tool and means to information repository necessary to know about software processes, functions, and maintenance. Documentation also provides information about how to use the product. Training in an attempt to improve the current or future employee performance by increasing an employee's ability to work through learning, usually by changing his attitude and developing his skills and understanding.

Stage-6: Deployment and Maintenance of Products

After detailed testing, the conclusive product is released in phases as per the organization's strategy. Then it is tested in a real industrial environment. It is important to ensure its smooth performance. If it performs well, the organization sends out the product as a whole. After retrieving beneficial feedback, the company releases it as it is or with auxiliary improvements to make it further helpful for the customers. However, this alone is not enough. Therefore, along with the deployment, the product's supervision.

Software Development Life Cycle Models

1. Waterfall Model

It is the fundamental model of the software development life cycle. This is a very simple model. The waterfall model is not in practice anymore, but it is the basis for all other SDLC models. Because of its simple structure, the waterfall model is easier to use and provides a tangible output. In the waterfall model, once a phase seems to be completed, it cannot be changed, and due to this less flexible nature, the waterfall model is not in practice anymore.

2. Agile Model

The agile model in SDLC was mainly designed to adapt to changing requests quickly. The main goal of the Agile model is to facilitate quick project completion. The agile model refers to a group of development processes. These processes have some similar characteristics but also possess certain subtle differences among themselves.

3. Iterative Model

In the Iterative model in SDLC, each cycle results in a semi-developed but deployable version; with each cycle, some requirements are added to the software, and the final cycle results in the software with the complete requirement specification.

4. Spiral Model

The spiral model in SDLC is one of the most crucial SDLC models that provides support for risk handling. It has various spirals in its diagrammatic representation; the number of spirals depends upon the type of project. Each loop in the spiral structure indicates the Phases of the Spiral model**.**

5. V-Shaped Model

The V-shaped model **in SDLC** is executed in a sequential manner in V-shape. Each stage or phase of this model is integrated with a testing phase. After every development phase, a testing phase is associated with it, and the next phase will start once the previous phase is completed, i.e., development & testing. It is also known as the verification or validation model.

20.Describe about agile modeling in details.

Ans: **The Agile Model** was primarily designed to help a project adapt quickly to change requests. So, the main aim of the Agile model is to facilitate quick project completion. To accomplish this task, it's important that agility is required. Agility is achieved by fitting the process to the project and removing activities that may not be essential for a specific project.

Also, anything that is a waste of time and effort is avoided. The Agile Model refers to a group of development processes. These processes share some basic characteristics but do have certain subtle differences among themselves.

Steps in the Agile Model

The Agile Model is a combination of iterative and incremental process models. The phases involve

1. Requirement Gathering
2. Design the Requirements
3. Construction / Iteration
4. Testing / Quality Assurance
5. Deployment
6. Feedback

Agile Model Steps

1. Requirement Gathering

In this step, the development team must gather the requirements, by interaction with the customer. development team should plan the time and effort needed to build the project. Based on this information you can evaluate technical and economical feasibility.

- Meet with the customer to really understand their needs and what they expect from the software.

- Identify the key requirements and business goals to make sure everyone is on the same page.
- Estimate how much time and effort it will take to develop the software.
- Assess if the project is technically possible and whether it's worth the investment from both a technical and economic standpoint.

2. Design the Requirements

In this step, the development team will use user-flow-diagram or high-level UML diagram to show the working of the new features and show how they will apply to the existing software. Wireframing and designing user interfaces are done in this phase.

- **Designing the system**: Once the requirements are gathered, the next step is to design the system's overall architecture based on those needs. This helps to verify the software is structured in a way that meets the user's expectations.
- **Creating wireframes**: Next, wireframes for the user interface (UI) are created. These are simple blueprints that show how the software will look and how users will interact with it, ensuring it's user-friendly and easy to navigate.
- **High-level design with UML diagrams**: At this stage, high-level designs using UML (Unified Modeling Language) diagrams are created to visually represent the software's structure and how different parts will work together.
- **Prototyping for feedback**: Prototypes are made to give stakeholders an early look at the software. This helps gather feedback early in the process and allows for adjustments before the full development begins.

3. Construction / Iteration

In this step, development team members start working on their project, which aims to deploy a working product. Each cycle typically consist between **1-4 weeks**, and at the end, the team delivers a working version of the software.

- **Development of Features**: The team works on the features identified during the requirement and design phases.
- **Coding and Implementation**: New functionalities are coded and integrated into the software based on the goals for that specific iteration.
- **Delivering a Working Product**: After each iteration, a usable version of the software is ready.
- **Incremental Improvement**: With every cycle, the software is enhanced, adding more features and refining existing ones.

4. Testing / Quality Assurance

Testing involves Unit Testing, Integration Testing, and System Testing, Which help in the agile development models:

- Unit testing is the process of checking small pieces of code to ensure that the individual parts of a program work properly on their own. Unit testing is used to test individual blocks (units) of code.
- Integration testing is used to identify and resolve any issues that may arise when different units of the software are combined.
- Goal is to ensure that the software meets the requirements of the users and that it works correctly in all possible scenarios.

5. Deployment

In this step, the development team will deploy the working project to end users.

Once an iteration is finished and fully tested, the software is ready to be released to the end users. In Agile, deployment isn't a one-time event—it's an ongoing process. Updates and improvements are rolled out regularly, making sure the software keeps evolving and getting better with each release.

- Deploy the software to a test or live environment so that it can be used by customers or end-users.
- Make the software accessible to users, verifying they can start using it as expected.
- Verify the deployment goes smoothly and fix any issues that come up quickly

6. Feedback

This is the last step of the Agile Model. In this, the team receives feedback about the product and works on correcting bugs based on feedback provided by the customer.

- Take feedback from customers, users, and stakeholders after each iteration.
- Understand how well the product meets user needs and identify areas for improvement.
- Check for bugs or issues that need fixing.
- Make adjustments to the development plan based on feedback to improve the product further.

21.Discuss the Extreme Programming Process.

Ans: Extreme Programming (XP) is an Agile software development methodology that focuses on delivering high-quality software through frequent and continuous feedback, collaboration, and adaptation. XP emphasizes a close working relationship between the development team, the customer, and stakeholders, with an emphasis on rapid, iterative development and deployment. Extreme Programming (XP)

Agile development approaches evolved in the 1990s as a reaction to documentation and bureaucracy-based processes, particularly the waterfall approach. Agile approaches are based on some common principles, some of which are:

1. Working software is the key measure of progress in a project.
2. For progress in a project, therefore software should be developed and delivered rapidly in small increments.
3. Even late changes in the requirements should be entertained.
4. Face-to-face communication is preferred over documentation.
5. Continuous feedback and involvement of customers are necessary for developing good-quality software.
6. A simple design that involves and improves with time is a better approach than doing an elaborate design up front for handling all possible scenarios.
7. The delivery dates are decided by empowered teams of talented individuals.

Extreme programming is one of the most popular and well-known approaches in the family of agile methods. an XP project starts with user stories which are short descriptions of what scenarios the customers and users would like the system to support. Each story is written on a separate card, so they can be flexibly grouped.

Good Practices in Extreme Programming

Some of the good practices that have been recognized in the extreme programming model and suggested to maximize their use are given below:

Extreme Programming Good Practices

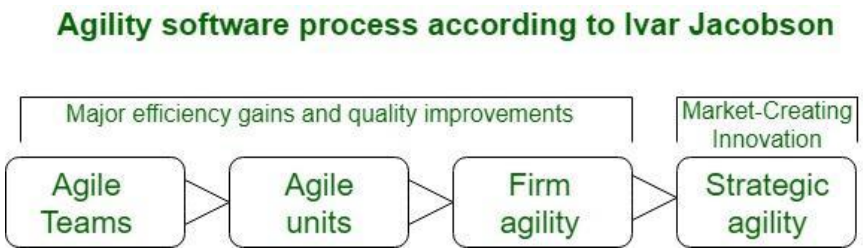
- **Code Review:** Code review detects and corrects errors efficiently. It suggests pair programming as coding and reviewing of written code carried out by a pair of programmers who switch their work between them every hour.
- **Testing:** Testing code helps to remove errors and improves its reliability. XP suggests test-driven development (TDD) to continually write and execute test cases. In the TDD approach, test cases are written even before any code is written.
- **Incremental development:** Incremental development is very good because customer feedback is gained and based on this development team comes up with new increments every few days after each iteration.
- **Simplicity:** Simplicity makes it easier to develop good-quality code as well as to test and debug it.
- **Design:** Good quality design is important to develop good quality software. So, everybody should design daily.
- **Integration testing:** helps to identify bugs at the interfaces of different functionalities. Extreme programming suggests that the developers should achieve continuous integration by building and performing integration testing several times a day.

22.What is agility in the context of software engineering work?

Ans: Agility has become today's buzzword when describing a contemporary software method. Everyone is agile. An associate agile team could be a nimble team able to befittingly reply to changes. Modification is what software development is extremely abundant.

- Changes within the software being engineered,
- Changes to the team members,
- Changes attributable to new technology,
- Changes of all types will have an effect on the merchandise they build or the project that makes the merchandise.

All changes can be represented as shown in the below diagram which is considered according to Ivar Jacobson Agility process of Software.



Agility - Software Engineering

Support for changes ought to be inherent in everything we tend to kill software, one thing we tend to embrace as a result of it's the guts and soul of software. Associate in agile teams acknowledges that software is developed by people operating in groups the talents of those folks, and their ability to collaborate are at the core for the success of the project. In Jacobson's read, the generality of modification is that the primary driver for agility. Software engineers should be fast on their feet if they're to accommodate the speedy changes that Jacobson describes. But agility is over an efficient response to alter.

- It encourages team structures and attitudes that create communication (among team members, between technologists and business folks, between software engineers and their managers) additional facile.

- It emphasizes speedy delivery of operational software Associate in emphasizes the importance of intermediate work merchandise (not continuously a decent thing);
- It adopts the client as a vicinity of the event team and works to eliminate the "us and them" angle that continues to perforate several software projects;
- It acknowledges that coming up within an unsure world has its limits which a project arrange should be versatile.

Agility is applied to any software method. However, to accomplish this, it's essential that the method be designed during a manner that enables the project team to adapt tasks and to contour them, conduct coming up within a good manner that understands the fluidity of an agile development approach, eliminate about the foremost essential work products and keeps them lean, Associate in emphasize a progressive delivery strategy that gets operating package to the client as apace as possible for the merchandise sort and operational atmosphere.

23.(i) Compare the life cycle models based on their distinguishing factors, strengths and weaknesses.(7)

Waterfall model	Spiral model	Prototyping model	Incremental model
Requirements must be clearly understood and defined at the beginning only.	The **requirements** analysis and gathering can be done in iterations because requirements get changed quite often.	**Requirements analysis** can be made in the later stages of the development cycle, because requirements get changed quite often.	The **requirements analysis** can be made in the later stages of the development cycle.
The **development team** having the adequate experience of working on the similar project is chosen to work on this type of process model	The **development team** having less experience of working on the similar projects is allowed in this process model	The **development team** having less experience of working on the similar projects is allowed in this process model.	The **development team** having the adequate experience of working on the similar project is chosen to work on this type of process model.
There is **no user involvement** in all the phases of development process.	There is **no user involvement** in all the phases of development process.	There is **user involvement** in all the phases of development process.	There is **user involvement** in all the phases of development process.
When the requirements are reasonably well defined and the development effort suggests a purely linear effort then the **waterfall model** is **chosen.**	Due to iterative nature of this model, the risk identification and rectification is done before they get problematic. Hence for handling real time problems the **spiral model is chosen.**	When developer is unsure about the efficiency of an algorithm or the adaptability of an operating system then the **prototyping model is chosen.**	When the requirements are reasonably well defined, the development effort suggests a purely linear effort and when limited set of software functionality is needed quickly then the **incremental model is chosen.**

(ii)Discuss the prototyping model, what is the effect of designing prototype on the overall cost of the software project?(6)

Ans: The Prototyping Model in software development involves creating a working model

(prototype) of a system to explore design, user feedback, and refine requirements. While designing a prototype adds initial costs, it often reduces the overall project cost by catching and fixing issues early in the process, preventing costly rework later.

Prototyping may have some initial costs of developing, but it reduces the overall budget by helping your product to be free of the errors or glitches that could have occurred if the idea was made from scratch without any prior user testing.

Furthermore, prototyping also helps to understand the intrinsic flaws, shortcomings, and drawbacks that can be improved during the product development process.

If the prototyping process is ignored completely, it might result in the restructuring and redesigning of the entire product after spending all your resources on its development. So, the effect of designing a prototype on the overall cost of a software project is to reduce the additional costs of restructuring and reframing it after its full-fledged development- which might cost a fortune.

24.Explain the component based software development model with a neat sketch.

Ans: The component-based assembly model uses object-oriented technologies. In object-oriented technologies, the emphasis is on the creation of classes. Classes are the entities that encapsulate data and algorithms. In component-based architecture, classes (i.e., components required to build application) can be uses as reusable components. This model uses various characteristics of spiral model. This model is evolutionary by nature. Hence, software development can be done using iterative approach. In CBD model, multiple classes can be used. These classes are basically the prepackaged components. The model works in following manner:

Step-1: First identify all the required candidate components, i.e., classes with the help of application data and algorithms.

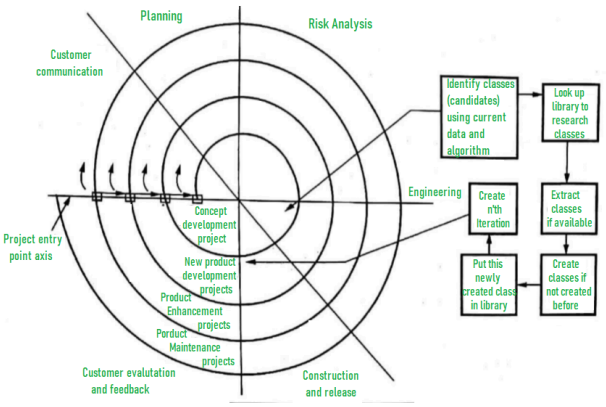
Step-2: If these candidate components are used in previous software projects then they must be present in the library.

Step-3: Such preexisting components can be excited from the library and used for further development.

Step-4: But if the required component is not present in the library then build or create the component as per requirement.

Step-5: Place this newly created component in the library. This makes one iteration of the system.

Step-6: Repeat steps 1 to 5 for creating n iterations, where n denotes the number of iterations required to develop the complete application.



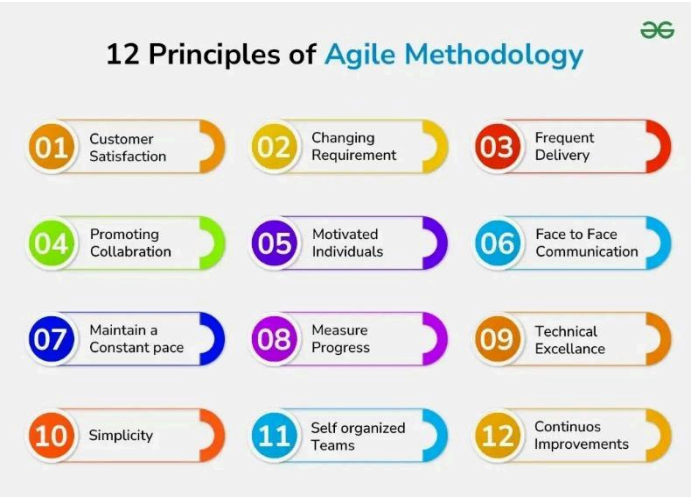
Characteristics of Component Assembly Model:

- Uses object-oriented technology.
- Components and classes encapsulate both data and algorithms.
- Components are developed to be reusable.
- Paradigm similar to spiral model, but engineering activity involves components.
- The system produced by assembling the correct components.

25.List the principles of agile software development.

Ans: **Agile Principles**

There are 12 agile principles mentioned in the Agile manifesto. Agile principles are guidelines for flexible and efficient software development. They emphasize frequent delivery, embracing change, collaboration, and continuous improvement. The focus is on delivering value, maintaining a sustainable work pace, and ensuring technical excellence.



The Agile Alliance defines twelve lightness principles for those who need to attain agility:

1. Our highest priority is to satisfy the client through early and continuous delivery of valuable computer software.
2. Welcome dynamic necessities, even late in development. Agile processes harness modification for the customer's competitive advantage.
3. Deliver operating computer software often, from a pair of weeks to a couple of months, with a preference to the shorter timescale.
4. Business individuals and developers should work along daily throughout the project.
5. The build comes around actuated people. offer them the setting and support they have, and trust them to urge the task done.
6. the foremost economical and effective methodology of conveyancing info to and among a development team is face-to-face speech.
7. Working with computer software is the primary life of progress.
8. Agile processes promote property development. The sponsors, developers, and users will be able to maintain a relentless pace indefinitely.
9. Continuous attention to technical excellence and smart style enhances nimbleness.
10. Simplicity—the art of maximizing the number of work not done—is essential.
11. the most effective architectures, necessities, and styles emerge from self–organizing groups.
12. At regular intervals, the team reflects on a way to become simpler, then tunes and adjusts its behavior consequently.